Assignment 3: Agents

In this assignment we will gradually build an interactive LLM agent that executes shell commands on behalf of a user.

The agent will be structured as a command line programmed implemented in python and called `doit`, which takes the user request as a commandline parameter.

The agent will execute bash or zsh commands. These are very common shells for Mac and Linux. They can also be installed on Windows through the WSL or the GitBash project. But you are highly advised to transition to Mac or Linux.

Single command at a time.

Write a CLI (command-line interface) program called `doit` that:

1. takes in an instruction in natural language;
2. translates it to a shell command in either `bash` or `zsh` (you decide which one, according to your shell) and prints the command;
3. executes the command in the shell, and writes its output to the screen.

For example:

```
> doit "list the files in my Documents folder"

ls ~/Documents

doc1.pdf
doc2.pdf
doc3.pdf
```

The translation should be done with an LLM call. Use an API call to a hosted model provider.

Things to think about:

- How would you capture the model's command output? need to request it in a convenient format.
- What happens if the user requests something that is not possible to do in the shell? (what would be an appropriate response)
- What happens if the user asks something like "tell me a joke" or "what can you do"? you should respond nicely to such requests.

One way to start is to separate the model's response from the execution step. The model may need to produce a shell command, a regular answer, or an explanation that the request cannot be done as a shell command. Your code then has to recognize which case it got and handle it.

Despite having no file extension, `doit` should be written in python.

It should be added to your path and work when invoked from within any directories in your system.

In Python, a simple way to execute a command is with `subprocess`:

```
import subprocess

def run_shell(command: str, shell: str = "/bin/bash"):
    result = subprocess.run(
        command,
        shell=True,
        executable=shell,
        text=True,
        capture_output=True,
        timeout=20,
    )
    return {
        "stdout": result.stdout,
        "stderr": result.stderr,
        "returncode": result.returncode,
    }
```

You may use this code or modify it. Make sure you capture `stdout`, `stderr`, and the return code. If you work on Windows, use WSL or Git Bash so that the commands are still Linux-style `bash` / `zsh` commands.

Think of how to implement it. You can either use the LLM-provider's built-in tool calling mechanism, or roll your by providing tool descriptions and tool-calling instructions in the system prompt. Do whatever is convenient. Note that you generally do not want to implement each command such as `ls`, `grep`, `git` etc as separate tools.

Identifying dangerous commands

Now change the behavior of the program. Commands that only show information, like `ls` or `grep`, should be executed directly. But commands that modify the filesystem (create files, move files, delete files, etc) should not be executed. Instead, the program should show the command to the user, explain what it does, and ask the user if they want to proceed. Only when the user types `y` the command will be executed.

Think about:

- How will you identify these cases? one option would be to do it through a separate LLM call.

Model flexibility

Now extend the program to work also with a local model, that runs on your machine. The prerequisite is to run local models around 4B. Students with stronger computers may use larger models around 7B-8B.

Medium models, around 4B:

- `qwen3:4b-instruct` - modern model with tool/structured-output support.
- `gemma3:4b` - capable general instruction model, not specifically a tool-calling model.

Larger models, around 7B-8B:

- `mistral:7b` - supports function calling.
- `llama3:8b` - strong general instruction model, but not specifically a tool-calling model.

Possible local serving options:

- Ollama: easiest command-line setup, for example `ollama pull qwen3:4b-instruct`.
- LM Studio: convenient GUI and local OpenAI-compatible server.
- llama.cpp server: more control, but more setup.

These models differ in quality and in how well they follow structured-output or tool-use instructions. Part of the assignment objective is to experiment with these differences.

Specifically, you must explore using all three models for all your experimentation and test cases:

- one LLM-provider API model
- one local instruction model without tool-calling support
- one local instruction model with tool-calling support

Use a file called `doit.cfg` located in the home directory to determine which model to call.

Use the `LiteLLM` package to call the model, and make it easy to switch between different model providers.

ACDL documentation

Throughout the assignment, document the context sent to the LLM using ACDL. Do this while you work on your implementation, not only at the end when writing the report. Use the ACDL documentation: https://acdlang26.github.io/acdlsite/syntax-reference.html

Your report should include, for each version of the agent, both the ACDL descriptions of all relevant contexts, as well as the specific prompts (prompt templates) that you used. For ACDL, include both the textual description, as well as the generated visual representation.

Every invocation of `doit` is considered as its own turn.

Multi-turn

Let's move from isolated requests to turns that depend on each other. We want to support commands to refer to previous interactions.

For example, after the invocation `doit "list the files in my Documents folder"` we may follow up with `doit "now sort them by date"` and then `doit "no, i meant latest first"` or `doit "i meant creation date"`.

Note that each call is still a new and separate invocation of the `doit` process, but there is information that is retained between them. One way to do it is to keep a file with the information that has to persist between invocations. A common way is to create a hidden folder called `.doit` or similar in your home folder, and put all these state files there. You are of course free to use whatever you want.

Think about:

- how and where do you store the history?
- how do you present the history to the LLM?
- need to distinguish between new commands, and commands that refer to previous ones. When referring to previous commands, need to know which one.

Clarifications

Add the ability for the program to ask the user when it is not sure. Eg:

```
> doit "list the files in my home folder sorted by date"

Do you want to sort by:
1. creation date
2. access date
```

The program will then wait for an answer before continuing.

Think about:

- how do you identify uncertainties? you want the feature to be useful but not annoying, asking only when "needed"
- what do you do if the user does not answer the question?
- how do you implement this mechanism? If you use tool calling, the question can be represented as a tool.

Richer interactions

Allow the program to answer also non-commands, such as “how do i do X”. these should also work with clarifications etc as needed, but the end result should be an answer, and not an invocation. Followup like “modify it to do y” and “execute it” etc, should also work. Each followup is a new invocation of `doit`

Think about:

- how to make it work?
- what can happen in longer sequences of interaction?

Memory

Allow the program to store “memories” about the user upon request. This is different from multi-turn history: a memory should persist even if the user opens a new terminal window, launches the program from another directory, or starts a new session later. In later interactions, the program should be aware of the memory and act accordingly.

Note that the same command may trigger both an action and a memory store. For example, `doit "move to ~/school/llms/ass3. this is my LLM class project folder."` should change the current working context and also remember that this folder is the user’s class project folder. Later, `doit "go to my llm class project"` should navigate to the correct folder based on the memory.

Think about:

- how are memories stored and when?
- how do memories appear in the LLM context?
- what about commands that refer to previous memories like “I changed my mind about the sorting order, ask me each time”?

User awareness

Currently the program is aware of its own actions, but not of the user’s actions. Make it aware also of user’s actions. Shells like bash have a mechanism that allows them to store command history. Use it.

For example, if the user manually runs `cd ~/school/llms/ass3`, `mkdir data`, and `python train.py` in the shell, and then asks `doit "summarize what I just did"`, the program should be able to inspect recent shell history and distinguish between commands run by the user and commands run by `doit`.

Similarly, if the user manually runs `cd ~/school/llms/ass3` and then asks `doit "make a new folder called experiments"`, the program should understand the current directory and recent user actions, not only its own previous actions.

There are multiple reasonable ways to implement this. You may read shell history, track the current working directory, add shell hooks, or use another approach. Changes outside your program, such as adding a small `.bashrc` or `.zshrc` snippet, are allowed, but you must document what you changed and why.

Think about:

- how to integrate the user's behavior information in the context?
- how to integrate user's behaviors and agent behavior in the context?

Output awareness

If not supported already, make sure it is possible to ask questions directly about the shell output. Such questions can be answered either by the LLM directly, or by invocation of further shell commands.

For example, after `doit "list the largest files in this directory"`, the user may ask `doit "which of these looks safe to delete?"` or `doit "why did that command fail?"`. The program should have enough access to the previous command, its output, and its error messages to answer or continue working from them.

Multi-tasking

It is possible for a user to work in two terminal windows while switching between them. For example, I may work on coding in one folder, and on report writing in another folder. In each of these, I may issue `doit` to do stuff. This results in some possible complications, because now references to history should be aware of where the command is issued from.

For example, consider this sequence:

Window 1:

```
> doit "move to my llm class folder"
ok
> doit "list the files"
[file listing]
```

Window 2:

```
> doit "move to my documents folder"
ok
> doit "create a folder for each year from 2020 to 2026"
ok
```

Window 1:

```
> doit "sort them by date"
```

This should refer to the listed files, not to the created folders.

On the other hand:

Window 1:

```
doit "now do the same folder task we did in window 2 here"
```

Should create the yearly folders under the llm class folder.

Devise a way for the program to behave nicely in these situations. An important question here is how to make it aware of the context in which it is running, and that the history sequence it sees may be composed of several different streams.

One possible direction is to give each terminal window a session id. For example, a small `.bashrc` or `.zshrc` snippet can set an environment variable when the terminal starts, and every `doit` process launched from that terminal can read the same session id. The program can then keep separate history streams for different terminal windows, while still allowing references across streams when the user asks for them explicitly.

Further extensions

Think of three additional extensions to your agent. Describe all of them, and implement one.

The implemented extension should be a real agent capability, not only a UI change or a small convenience wrapper. It should make the agent better at planning, remembering, using tools, handling uncertainty, managing context, or recovering from mistakes.

Examples of possible extensions include:

- context compaction or summarization for long histories;
- multi-step tool use, where one user request can lead to a sequence of several shell commands or clarification steps;
- project profiles, with project memories (the equivalent of agent.md), so the agent can behave differently in different folders;
- command plans that explain what will happen before running several commands;

Your report should explain why you chose the extension, how it is implemented, and show at least one interaction where the extension matters.

Grading

This assignment is intentionally broad, so as not to restrict how you address and implement the different sections. The required behavior is that the agent works, but the implementation choices are yours to make.

The grade will be based on both the implementation and the report.

The implementation will be evaluated according to whether the agent actually works, how completely it implements the tasks above, and how well the different parts fit together. This includes command generation, shell execution, safety checks, model flexibility, multi-turn behavior, clarifications, richer interactions, memory, user-awareness, output-awareness, multi-tasking, and the additional extension you chose to implement.

The report will be evaluated according to how clearly you explain what you did, why you made your design choices, what worked, what did not work, and what the limitations of your system are. The report should make it easy to understand the behavior of your system.

ACDL is an important part of the grading. For each section, document the contexts, assumptions, decisions, and limitations that shaped your implementation. The ACDL should make it possible to compare your report, logs, prompts, memory/session state, and code behavior. If the implementation works in a demo but the relevant contexts and design decisions are not documented, it will be harder to evaluate and will result in a lower grade.

A strong submission should implement all parts of the assignment correctly. A great submission will usually be distinguished by the quality of the harder agentic parts: rich memory, awareness of the user's shell history, multi-terminal/session handling, and the additional extension. In these parts, we will look for careful context management, clear separation between different sessions and directories, good use of logs, and behavior that continues to make sense in longer or ambiguous interactions.

What to submit?

Submit your code and a report.

In the report, for each section of the assignment:

- Describe what you implemented.
- Explain the design decisions you made.
- Include at least one example interaction that shows the behavior. Preferably, show several interesting ones.
- Discuss limitations, failures, or cases where the behavior is imperfect.

Also include:

- Interaction logs that show interesting behaviors, including both successful cases and failure or recovery cases.
- A comparison between two local models you used, preferably one that was trained or adapted for tool calling and one that was not. Include at least one interaction where the weaker model failed, behaved worse, or needed different prompting, and discuss what this showed about structured outputs, tool-use decisions, clarifications, or error handling.
- The prompt templates, tool definitions, structured-output formats, or schemas used by your system.